



密码学引论实验报告

姓名: 江李阳, 王健一, 邱泽辉

学号: 202400460042, 202400460037, 202400460046

专业: 密码科学与技术

日期: May 14, 2026

Contents

1	实验目的	2
2	实验环境	2
3	实验原理	2
3.1	密文分组链接模式	3
3.2	CBC 模式解密	3
3.3	明文填充	3
3.3.1	PKCS#5 填充标准	3
3.3.2	PKCS#7 填充标准	3
3.4	Padding oracle 攻击	4
3.5	攻击本质	5
4	目标密文与分组	5
4.1	密文分组	5
5	攻击算法设计	6
5.1	整体攻击思路	6
5.2	基础版算法的主线	6
5.3	按块恢复的执行过程	7
5.4	实际提交密文格式	7
5.5	算法复杂度	7
6	代码实现说明	7
6.1	程序结构	7
6.2	主要实现流程	8
6.3	实现体会	8
7	运行结果与破解明文	8
7.1	分组恢复结果	9
7.2	完整明文恢复结果	10
8	实验分析与总结	10
8.1	实验结果分析	10
8.2	实现过程中的体会	10
8.3	安全启示	11
9	附录	12
9.1	实验运行代码	12

1 实验目的

1. 理解 AES-128-CBC 模式的加解密结构，明确明文分组、密文分组和初始向量 IV 之间的关系；
2. 掌握 PKCS#7 填充规则，理解填充合法性判断如何泄露明文相关信息；
3. 编写 Padding Oracle 攻击程序，利用服务器返回的填充状态恢复 CBC 解密过程中的中间状态；

2 实验环境

本实验在本地编写攻击程序，并通过访问实验指导书给出的解密服务器完成 Padding Oracle 攻击。实验过程中，攻击者不需要知道 AES 密钥，只需依据服务器返回的状态判断解密结果的 PKCS#7 填充是否合法。

本小组成员学号末位分别为 2、7、6，因此目标密文编号按下式确定：

$$2 + 7 + 6 = 15 \equiv 5 \pmod{10}$$

故本实验选择 5 号目标密文进行攻击。

Table 1: 实验环境

项目	配置或说明
操作系统	Windows 11
编程语言	Python 3
开发工具	Visual Studio Code
加密算法	AES-128-CBC
填充标准	PKCS#7
目标密文编号	5 号密文
密钥组	Key_2
解密接口	http://10.102.33.67:8208/dec_2?data=

实验指导书中的解密服务器会对输入密文进行 AES-128-CBC 解密，并检查解密后明文是否满足 PKCS#7 填充规则。若填充正确，服务器返回 HTTP 200；若填充错误，则返回 HTTP 500。因此，本实验通过构造查询密文并观察服务器反馈状态，逐步恢复目标密文对应的明文。

实验材料中给出的 5 号密文属于 Key_2 加密所得密文，因此程序访问的解密接口为 dec_2。实验运行时需要保证本机能够访问学校实验服务器，通常需要处于校园网环境或连接学校 VPN。

3 实验原理

本实验的核心原理是 Padding Oracle 攻击。该攻击针对采用 CBC 工作模式并使用 PKCS#7 填充的分组密码系统。当解密服务器会向攻击者反馈“填充是否合法”时，即使攻击者不知道 AES 密钥，也可以通过构造密文并反复查询服务器，逐字节恢复目标密文对应的明文。

3.1 密文分组链接模式

CBC (Cipher Block Chaining, 密文分组链接) 模式的基本思路是: 加密当前明文分组时, 先将它与前一密文分组异或, 再送入分组密码算法。设明文分组为 $m_1, m_2, \dots, m_s$, 密文分组为 $c_1, c_2, \dots, c_s$, 初始向量为 IV , 并记 $c_0 = IV$, 则 CBC 加密关系为

$$c_i = \text{Enc}_k(m_i \oplus c_{i-1}), \quad i = 1, 2, \dots, s.$$

由此可以看出, 前一密文分组会影响后一分组的加密结果。例如, m_1 不仅影响 c_1 , 还会通过链式关系继续影响后续分组。因此 CBC 模式具有明显的前后依赖关系, 加密时不能像 ECB 模式那样对所有明文分组完全独立地并行处理。

3.2 CBC 模式解密

由 CBC 模式的加密关系可知, 解密时必须使用 $c_0 = IV$, 因此实际传输和保存的密文通常写作

$$IV \parallel c_1 \parallel c_2 \parallel \dots \parallel c_s.$$

设密文分组 c_i 对应的明文分组为 p_i , 则有

$$p_i = \text{Dec}_k(c_i) \oplus c_{i-1}, \quad i = 1, 2, \dots, s,$$

其中 $c_0 = IV$ 。将 $\text{Dec}_k(c_i)$ 记作中间状态 A_i , 则上式可以写成

$$p_i = A_i \oplus c_{i-1}. \quad (1)$$

因此, 只要能够恢复出中间状态 A_i , 再与真实前一密文分组 c_{i-1} 异或, 就可以得到当前密文分组对应的真实明文分组 p_i 。Padding Oracle 攻击的关键就在于如何通过服务器反馈逐步恢复 A_i 。

3.3 明文填充

AES 的标准分组长度为 128 bit, 即 16 字节, 而实际待加密明文 m 的长度不一定正好是 16 字节的整数倍。因此在加密前, 需要先对明文进行填充 (padding), 使填充后的长度成为分组长度的整数倍。

3.3.1 PKCS#5 填充标准

PKCS#5 以 8 个字节为一个分组。若明文长度不足一个分组, 则补足相应数量的字节; 若明文长度刚好等于一个完整分组, 也要额外补充一个新的分组。填充值等于填充字节数。例如, 若需要填充 7 个字节, 则补入 7 个 0x07; 若刚好是一个分组, 则补入 8 个 0x08。

3.3.2 PKCS#7 填充标准

PKCS#7 的规则与 PKCS#5 类似, 但分组长度不再限制为 8 字节, 而是适用于 1 ~ 255 字节的分组。对于 AES-128 而言, 一个分组为 16 字节。若某个明文分组还差 3 个字节补满, 则填充后的结果形如

$$M_{1 \sim 13}, 0x03, 0x03, 0x03.$$

因此, 解密服务器在解密完成后, 会检查最后若干字节是否满足 PKCS#7 填充规则。Padding Oracle 攻击正是利用了这个检查结果是否可区分这一点。

3.4 Padding oracle 攻击

Padding Oracle 攻击是一种选择密文攻击。设最后一个目标密文分组为 c_s ，攻击者先取一个 16 字节分组

$$r = r_1 r_2 \cdots r_{16}$$

作为伪造的前一分组，并将它与 c_s 一起提交给解密服务器。实验指导书中的基本形式是 $r \parallel c_s$ ；而在本实验环境中，为了满足服务器接口格式，程序实际会保留必要前缀，使目标块仍然处在整段提交密文的最后一个分组位置。无论前面前缀如何，服务器最终检查的仍然是

$$Dec_k(c_s) \oplus r = A_s \oplus r$$

这一分组的填充是否满足 PKCS#7 规则。

服务器会给出如下反馈：

- 如果不满足填充规则，则返回填充错误信息（HTTP 500）；
- 如果满足填充规则，则返回正确信息（HTTP 200）。

例如，若某次查询后最后一个明文分组的末尾恰好是 3 个 0x03，则服务器会认为填充合法。攻击者正是根据这种反馈，逐步反推出中间状态 A_s 。

- 判断原始填充长度：在本实验的基础实现中，先从真实前一密文分组出发，逐个修改其字节并观察服务器反馈。假设最后一个明文分组原本的填充长度为 3，那么修改 r_1 到 r_{13} 时，不会影响末尾 3 个填充字节，服务器仍会返回正确；而当修改到 r_{14} 时，就会破坏第一个填充字节，此时服务器返回错误。于是可以判断原始填充长度为 3。其示意如下：

c_s	data	...	padding	padding	padding
r	r_1	...	r_{14}	r_{15}	r_{16}
$\oplus$ 后结果	data	...	0x03	0x03	0x03

若修改 r_j 后首次返回错误，则可判断原始填充长度为 $16 - j + 1$ 。

- 利用原始填充恢复部分中间状态：假设根据上一步判断出填充长度为 t ，则末尾 t 个字节满足

$$a_{16-t+1} \oplus r_{16-t+1} = t, \quad \cdots, \quad a_{16} \oplus r_{16} = t.$$

因而可以直接得到

$$a_{16-t+1} = r_{16-t+1} \oplus t, \quad \cdots, \quad a_{16} = r_{16} \oplus t.$$

这样便先恢复出了中间状态末尾的一段后缀。

- 进一步恢复前一字节：假设当前已经恢复出 $a_j, a_{j+1}, \cdots, a_{16}$ ，则可以调整伪造分组的后缀，使得这些位置在异或后全部变成新的目标填充值。例如，若当前想把末尾构造造成 0x02 0x02，则令

$$r'_{16} = a_{16} \oplus 0x02.$$

接着枚举 r'_{15} 的取值，并不断提交查询，直到服务器返回填充正确。此时有

$$a_{15} \oplus r'_{15} = 0x02,$$

从而得到

$$a_{15} = r'_{15} \oplus 0x02.$$

用同样的方法继续向前推进，就可以依次恢复 $a_{14}, a_{13}, \dots, a_1$ 。

- 恢复真实明文：当最后一个目标分组 c_s 的全部中间状态都恢复完成后，便得到

$$A_s = a_1 a_2 \cdots a_{16}.$$

再结合真实前一密文分组 c_{s-1} ，利用公式

$$p_s = A_s \oplus c_{s-1}$$

即可恢复出该分组对应的真实明文。对其他目标密文分组重复这一过程，就可以逐块还原完整原始明文。

3.5 攻击本质

Padding Oracle 攻击并不是直接破解 AES 算法本身，也不是恢复 AES 密钥，而是利用了解密系统实现中的信息泄露。具体来说，服务器把“填充正确”和“填充错误”以不同的状态返回给攻击者，使攻击者能够通过大量选择密文查询，逐步推断 CBC 解密的中间状态，最终恢复明文。

因此，本实验说明：只提供机密性而缺少完整性保护的 CBC 加密实现是不安全的。在实际系统中，不能将可区分的 padding 错误直接暴露给外部用户，更安全的做法是使用认证加密模式，或者在解密前先进行消息认证。

4 目标密文与分组

本实验选择的目标密文编号为 5，对应的解密接口为 `dec_2`。目标密文采用 AES-128-CBC 模式加密，并使用 PKCS#7 填充。由于 AES 的分组长度为 16 字节，因此需要按照每 16 字节，即每 32 个十六进制字符，对目标密文进行分组。

4.1 密文分组

目标密文的整体结构为：

$$IV \parallel C_1 \parallel C_2 \parallel C_3$$

其中， IV 为初始向量， C_1, C_2, C_3 为三个密文分组。按照 16 字节分组后得到：

Table 2: 5 号目标密文分组结果

分组	十六进制表示
IV	46307250616464316e674f7261636c33
C_1	9ae0735429869542efc40dcdc3f4c170
C_2	649463f719c5ddf4ce8c6d1ef0e5d41a
C_3	c5d137629e3fe1340cfaad7e21d65d14

在 CBC 解密过程中, IV 可视为第 0 个密文分组, 即 $C_0 = IV$ 。因此三个明文分组的恢复关系分别为:

$$P_1 = Dec_k(C_1) \oplus IV, \quad (2)$$

$$P_2 = Dec_k(C_2) \oplus C_1, \quad (3)$$

$$P_3 = Dec_k(C_3) \oplus C_2. \quad (4)$$

后续攻击过程的目标就是分别恢复 $D_k(C_1)$ 、 $D_k(C_2)$ 和 $D_k(C_3)$ 的中间状态, 再与对应的前一分组异或, 得到当前密文分组对应的明文, 最终拼接出完整明文。由于 Padding Oracle 攻击总是针对“提交密文中的最后一个分组”进行判断, 因此在实际查询时, 需要构造密文使当前待恢复的目标块位于查询密文的末尾位置。

5 攻击算法设计

本实验的攻击算法按照实验指导书中的思路设计, 并与基础实现 `attack\_slow.py` 对应。第 3 节已经说明了 Padding Oracle 攻击的原理, 本节不再重复推导公式, 而是从算法执行流程的角度说明程序如何一步步恢复目标明文。

5.1 整体攻击思路

程序首先将目标密文拆分为

$$IV, C_1, C_2, C_3$$

四个分组, 然后从最后一个目标分组开始, 逐块恢复对应的中间状态和明文。由于最后一个分组最容易直接观察到原始填充信息, 因此程序先处理 C_3 , 再依次处理 C_2 和 C_1 。

5.2 基础版算法的主线

基础版程序按照“先判断原始填充长度, 再逐字节向前恢复”的思路进行。对单个目标分组而言, 主要步骤如下:

1. 取该目标分组真实前一密文分组作为初始参考分组;
2. 通过逐个修改参考分组的字节, 判断原始明文末尾的填充长度;
3. 根据已知填充长度, 先恢复中间状态末尾已经确定的若干字节;
4. 调整已恢复出的后缀, 使它们在解密后变成新的目标填充值;
5. 枚举当前待求字节, 直到服务器返回填充正确, 从而恢复新的中间状态字节;
6. 当整个中间状态恢复完成后, 再与真实前一密文分组异或, 得到目标明文分组。

这一设计与实验指导书中的推导是一致的, 也和第 3 节中的原理说明完全对应。

5.3 按块恢复的执行过程

对每个目标分组，程序都会先获得该分组对应的中间状态后缀，再逐步向前扩展。这个过程与原理部分的例子一致：先根据服务器反馈判断原始填充长度，再利用已知后缀去构造新的合法填充，逐字节恢复更多中间状态。等到整个中间状态都恢复完成后，再与真实前一密文分组异或，即可得到当前明文分组。

5.4 实际提交密文格式

理论上，Padding Oracle 攻击只需要让服务器检查 $R \parallel C_i$ 解密后的最后一个分组即可。但在本实验环境中，服务器接口接收的是完整密文串，因此程序实际提交的是带有必要前缀的密文。这样虽然提交格式比理论形式更长，但服务器最终检查的仍然是目标块对应的最后一个明文分组，因此不会改变攻击思路。

5.5 算法复杂度

对每个待恢复字节而言，最坏情况下需要枚举 256 种可能取值。每个 AES 分组有 16 个字节，因此恢复一个分组最多需要

$$16 \times 256 = 4096$$

次 oracle 查询。

本实验目标密文除 IV 外共有 3 个数据分组，因此理论上的查询上界为

$$3 \times 16 \times 256 = 12288.$$

实际运行时，并不是每个字节都会枚举满 256 次，因此真实查询次数通常小于该上界。与此同时，程序是按块逐步恢复的，因此一旦某个分组恢复完成，就可以立即得到对应的明文结果，便于随时检查程序是否运行正常。

6 代码实现说明

本节说明实验程序的实现思路。由于本实验的原理分析、攻击算法设计和运行结果都对应基础版程序，因此这里仅围绕 `attack\_slow.py` 进行说明。

6.1 程序结构

`attack\_slow.py` 的整体流程与第 3 节、第 5 节中的算法描述是一一对应的。程序先将目标密文拆成 IV, C_1, C_2, C_3 四个分组，然后从最后一个目标分组开始，逐块恢复中间状态和明文。

代码中用到的几个辅助函数分别完成如下工作：

- `get_bytes`: 从十六进制字符串中截取指定区间的字节；
- `get_list`: 将十六进制串按字节拆成列表；
- `list_to_str`: 将字节列表重新拼接成十六进制串。

这些辅助函数虽然比较简单，但使后面的分组截取、字节替换和伪造密文拼接都更加直观，也更接近实验推导时的写法。

6.2 主要实现流程

基础版程序的外层循环按照

$$C_3 \rightarrow C_2 \rightarrow C_1$$

的顺序逐块恢复明文。对每个目标分组，程序的主要步骤如下：

1. 取当前目标密文分组 y 及其真实前一密文分组 r ；
2. 先通过修改 r 的不同位置，判断原始填充长度；
3. 根据原始填充长度，先恢复中间状态末尾已经确定的若干字节；
4. 再不断调整伪造分组后缀，并枚举新的字节，逐步向前恢复整个中间状态；
5. 最后将恢复出的中间状态与真实前一密文分组异或，得到当前明文分组。

从实现方式上看，这个程序并没有把所有步骤压成过于抽象的函数，而是比较直接地按照攻击过程写出来，因此和实验指导书中的推导过程对应得比较紧，也更能体现“先判断填充长度，再继续向前恢复”的思路。

6.3 实现体会

从实际编写过程来看，Padding Oracle 攻击真正麻烦的地方不在公式，而在于如何把“逐字节构造、逐次提交、根据反馈修正”这些步骤准确地落到代码里。尤其是在基础版实现中，字节位置、填充值和中间状态之间的关系必须完全对应，否则后面恢复出来的结果就会整体出错。

虽然基础版程序运行速度较慢，但它把实验指导书中的思路保留得最完整，也最适合作为本实验报告正文的实现说明。

7 运行结果与破解明文

本节给出程序运行结果以及最终恢复得到的明文。实验开始时，先将目标密文拆分为 IV, C_1, C_2, C_3 四个分组，并验证原始密文能够通过服务器的 PKCS#7 填充检查。服务器返回填充正确，说明 oracle 接口可正常使用，后续攻击可以继续进行。

运行结果如下图：

```

second_experiment > exp.py > ...
5 def get_bytes(s,i,j):
9     return s[2*i:2*i+2*(j-i+1)]
10 def get_list(s):
11     lst = [s[i:i+2] for i in range(0, len(s), 2)]
12     return lst
13 def list_to_str(lst):
14     return ''.join(f'{x & 0xff:02x}' for x in lst)
15 p = []
16 for idx in trange(2,-1,-1):
17     y = cipher[idx+1]
18     r = cipher[idx]
19     pfx = 64*'0'
20     rb = 0
21     for i in trange(256):
22         c = pfx + r[:-2] + f'{i:02x}' + y
23         url = f"http://10.102.33.67:8208/dec_2?data={c}"
24         response = requests.get(url)
25         if 'HTTP 200' in response.text:

```

infinite@infiniteMacBook-Air ~/Documents/crypto_introduction > main ± /opt/homebrew/bin/pyt
hon3 /Users/infinite/Documents/crypto_introduction/second_experiment/exp.py
填充长度是 1
0% | 0/15 [00:00<?, ?it/s]
9% | 0/15 [00:00<?, ?it/s]
0% | 22/256 [00:00<00:03, 73.59it/s]
32% | 1/256 [00:00<00:09, 26.25it/s]
8% | 83/256 [00:00<00:02, 83.75it/s]
18% | 21/256 [00:00<00:03, 73.05it/s]
7% | 45/256 [00:00<00:02, 75.98it/s]
4% | 19/256 [00:00<00:05, 46.56it/s]
5% | 11/256 [00:00<00:03, 80.71it/s]
36% | 13/256 [00:00<00:03, 76.52it/s]
35% | 91/256 [00:01<00:02, 76.99it/s]
9% | 90/256 [00:01<00:02, 79.49it/s]
23% | 22/256 [00:00<00:04, 53.07it/s]
11% | 58/256 [00:00<00:02, 68.09it/s]
32% | 29/256 [00:00<00:03, 57.81it/s]
19% | 83/256 [00:01<00:02, 66.42it/s]
100% | 48/256 [00:00<00:04, 50.93it/s]
flag{555msdb6ax9bh0eqlvh1gfcx2cyr1ws666}
infinite@infiniteMacBook-Air ~/Documents/crypto_introduction > main ±

Figure 1: 代码运行结果

7.1 分组恢复结果

程序依次对三个目标密文分组进行恢复，最终得到的明文分组结果如表 3 所示。

Table 3: 各明文分组恢复结果

分组	明文十六进制表示	ASCII 表示
P_1	666c61677b353535 6d73646236617864	flag{555msdb6axd
P_2	3962683065716c76 683167666378326b	9bh0eqlvh1gfcx2k
P_3	6379723177733636 367d060606060606	cyr1ws666}

从表中可以看到，第三个明文分组的末尾为 06 06 06 06 06 06。这说明原始明文在最后补入了 6 个 PKCS#7 填充字节，因此这些字节不属于真实消息内容。

7.2 完整明文恢复结果

将三个明文分组顺次拼接后，可以得到带填充的完整明文，其十六进制表示为

```
666c61677b3535356d73646236617864
3962683065716c76683167666378326b
6379723177733636367d060606060606
```

去除末尾 6 个字节的 PKCS#7 填充后，真实明文的十六进制表示为

```
666c61677b3535356d73646236617864
3962683065716c76683167666378326b
6379723177733636367d
```

将其按 UTF-8 字符串解码，最终恢复得到 5 号目标密文对应的明文为

```
flag{555msdb6axd9bh0eqlvh1gfcx2kcyr1ws666}
```

这一结果与三个分组的逐块恢复结果是能够相互对应的，也说明程序恢复出的中间状态和最终明文是一致的。

8 实验分析与总结

从实验结果可以看出，Padding Oracle 攻击并不是去破解 AES 算法本身，而是利用了解密系统在实现上的信息泄露。攻击者虽然不知道密钥，但只要能够反复提交构造密文，并区分服务器返回的是“填充正确”还是“填充错误”，就可以逐步恢复 CBC 解密过程中的中间状态，最后还原出真实明文。

8.1 实验结果分析

本实验能够成功，依赖于以下几个条件同时成立：

- 目标系统使用了 CBC 分组模式，并采用 PKCS#7 填充；
- 解密服务器在解密后会检查填充是否合法；
- 攻击者能够区分服务器返回的不同错误状态。

这三个条件一旦同时满足，服务器实际上就成了一个“padding oracle”。虽然它没有直接把明文或密钥返回出来，但每一次反馈都在泄露一部分关于解密结果的信息。攻击者把这些信息不断累积起来，就能一点一点地恢复出完整明文。

8.2 实现过程中的体会

这次实验中比较明显的一点是：原理本身并不算太难，真正容易出错的是实现细节。例如，填充长度的判断、字节下标的对应关系、中间状态与真实明文的区分，这些地方只要错一个，后面的恢复过程就会全部受到影响。

8.3 安全启示

本实验说明：只有机密性、没有完整性保护的 CBC 加密实现是不安全的。为了防御 Padding Oracle 攻击，实际系统中通常要避免把可区分的 padding 错误直接暴露给外部用户，更安全的做法包括：

- 使用带认证的加密模式，如 GCM、CCM 等；
- 在解密前先做消息认证，而不是先解密再判断 padding；
- 对外统一错误返回，避免泄露“是 padding 错误还是其他错误”这类差异信息。

总的来说，这次实验不仅加深了对 CBC 模式和 PKCS#7 填充规则的理解，也更直观地说明了：密码算法本身安全，并不等于系统实现就一定安全。

9 附录

9.1 实验运行代码

```
1 import requests
2 from tqdm import *
3 ciphertext = '''46307250616464316e674f7261636c339ae0735429869542ef
4               c40dc3f4c170649463f719c5ddf4ce8c6d1ef0e5d
5               41ac5d137629e3fe1340cfaad7e21d65d14'''
6 cipher = [ciphertext[:32], ciphertext[32:64], ciphertext[64:96], ciphertext
7           [96:128]]
8 def get_bytes(s,i,j):
9     if j < i:
10         return ""
11     else:
12         return s[2*i:2*i+2*(j-i+1)]
13 def get_list(s):
14     lst = [s[i:i+2] for i in range(0, len(s), 2)]
15     return lst
16 def list_to_str(lst):
17     return ''.join(f'{x & 0xff:02x}' for x in lst)
18 p = []
19 for idx in range(2,-1,-1):
20     y = cipher[idx+1]
21     r = cipher[idx]
22     pfx = 64*'0'
23     rb = 0
24     for i in range(256):
25         c = pfx + r[:-2] + f'{i:02x}' + y
26         url = f"http://10.102.33.67:8208/dec_2?data={c}"
27         response = requests.get(url)
28         if 'HTTP 200' in response.text:
29             rb = i
30             print(rb)
31             break
32 padding_length = 1
33 if(idx != 2):
34     r = r[:-2] + f'{rb:02x}'
35 for j in range(14,-1,-1):
36     #修改rj,比如都加1
37     rj = (int(get_bytes(r,j,j),16) + 1) & 0xff
38     c = pfx + get_bytes(r,0,j-1) + f'{rj:02x}' + get_bytes(r,j+1,15) + y
39     url = f"http://10.102.33.67:8208/dec_2?data={c}"
40     response = requests.get(url)
41     if response.status_code == 500:
42         padding_length += 1
43     else:
44         print("填充长度是",padding_length)
45         break
```

```
45 #填充长度是k, 那么把a_{16-k}一直到a15还原出来
46 a = get_list(get_bytes(r,16-padding_length,15))
47 a = [int(a[i],16) ^ padding_length for i in range(len(a))]
48 for j in range(16-padding_length):
49     #修改r, 使得填充变成padding_length+1
50     rr = [a[i] ^ (padding_length+1) for i in range((len(a)))]
51     #搜搜r_{15-k}, 恢复a_{15-k}
52     rj_1 = 0
53     for i in range(256):
54         r_ = get_bytes(r,0,14-padding_length)+f'{i:02x}'+list_to_str(rr)
55         c = pfx + r_ + y
56         url = f"http://10.102.33.67:8208/dec_2?data={c}"
57         response = requests.get(url)
58         if 'HTTP 200' in response.text:
59             rj_1 = i
60             break
61     a = [rj_1 ^ (padding_length+1)] + a
62     padding_length +=1
63 r_n = get_list(r)
64 p = [int(r_n[i],16) ^ a[i] for i in range(len(a))] + p
65 for i in range(len(p)):
66     print(chr(p[i]),end='')
```